# Button組件

# 學習目標

- 本章節介紹:
  1. 建立一個Button
  2. Button外框效果
  3. Button顯示字串與圖示
  4. 設定Button大小
  5. 設定Button顏色
  6. Button元件放置
  7. 透過Button呼叫函式
  8. 設置Button字串位置
  9. 設置Button邊框粗細
  10. 設置Button狀態
  11. Button動態顯示
  12. 使Button消失

# 建立一個Button

- 語法1:

```
#設定顯示的字串
b1=Button(root,text="click!")
#建立按鈕
b1.pack()
```

- 語法2:

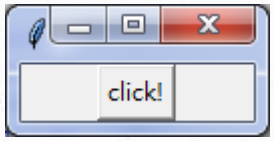
```
#設定顯示的字串且建立
Button(root,text="click!").pack()
```

# 建立一個Button

- 範例1:

```
from tkinter import *#載入函式庫
root = Tk()#建立一個視窗
#設定顯示的字串
b1=Button(root,text="click!")
#建立按鈕
b1.pack()
root.mainloop()#執行
```

- 執行結果:

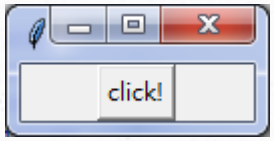


# 建立一個Button

- 範例2:

```
from tkinter import *#載入函式庫
root = Tk()#建立一個視窗
#設定顯示的字串
Button(root,text="click!").pack()
root.mainloop()#執行
```

- 執行結果:



# Button外框效果

- 語法:

```
#設定按鈕顯示字串與效果且建立按鈕
Button(root,text="click!",relief=FLAT).pack()
```

# Button外框效果

- 效果的指令：
  1. FLAT
  2. GROOVE
  3. RAISED
  4. RIDGE
  5. SOLID
  6. SUNKEN

# Button外框效果

- 範例:

```
from tkinter import *#載入函式庫
root = Tk()#建立一個視窗
#設定顯示的字串與效果且建立按鈕
Button(root,text="click!",relief=FLAT).pack()
root.mainloop()#執行
```

- 執行結果:

# Button顯示字串與圖示

- 語法:

```
#設定顯示的字串、圖示放置位置、圖示且建立按鈕
Button(root,text = "button",compound = "bottom",bitmap = "error").pack()
```

- 圖示顯示位置(compound)的指令：
  1. top
  2. bottom
  3. center
  4. left
  5. right

# Button顯示字串與圖示

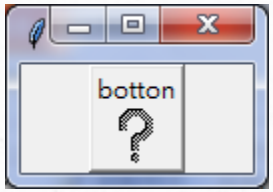
- 其他可用圖示(bitmap)的指令：
  1. error
  2. hourglass
  3. info
  4. questhead
  5. question
  6. warning
  7. gray12
  8. gray25
  9. gray50
  10. gray75

# Button顯示字串與圖示

- 範例:

```
from tkinter import *#載入函式庫
root = Tk()#建立一個視窗
#設定顯示的字串與圖示且建立按鈕
Button(root,text ="botton",compound = "bottom",bitmap =
"question").pack()
root.mainloop()#執行
```

- 執行結果:



# 設定Button大小

- 語法:

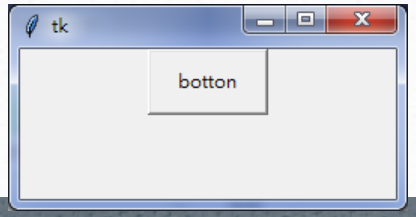
```
#設定顯示的字串與按鈕大小且建立按鈕
Button(root,text ="botton",width = 10,height = 2).pack()
```

# 設定Button大小

- 範例:

```
from tkinter import *#載入函式庫
root = Tk()#建立一個視窗
#設定顯示的字串與按鈕大小且建立按鈕
Button(root,text ="botton",width = 10,height = 2).pack()
#設定視窗大小+視窗放置的水平座標+視窗放置的垂直座標
root.geometry("250x100+1000+0")
root.mainloop()#執行
```

- 執行結果:



# 設定Button顏色

- 語法:

```
#設定顯示的字串、按鈕大小、背景顏色、前景顏色且建立按鈕
Button(root,text ="botton",width = 10,height = 2,bg="black",fg="white").pack()
```

# 設定Button顏色

- 範例:

```
from tkinter import *#載入函式庫
root = Tk()#建立一個視窗
#設定顯示的字串、按鈕大小與按鍵顏色且建立按鈕
Button(root,text ="botton",width = 10,height = 2,bg="black",fg="white").pack()
#設定視窗大小
root.geometry("250x100+1000+0")
root.mainloop()#執行
```

- 執行結果:

# Button元件放置

- 以下範例，介紹如何建立一個Button按鈕，您可以透過pack()方法，設定按鈕靠上、下、左、右其中一邊對齊。

- 語法:

```
#建立按鈕且靠左對齊
Button(root,text="click me!",width=10).pack(side="left")
##靠右對齊
Button(root,text="click me!",width=10).pack(side=" right ")
#靠上對齊
Button(root,text="click me!",width=10).pack(side=" top ")
#靠下對齊
Button(root,text="click me!",width=10).pack(side=" bottom ")
```

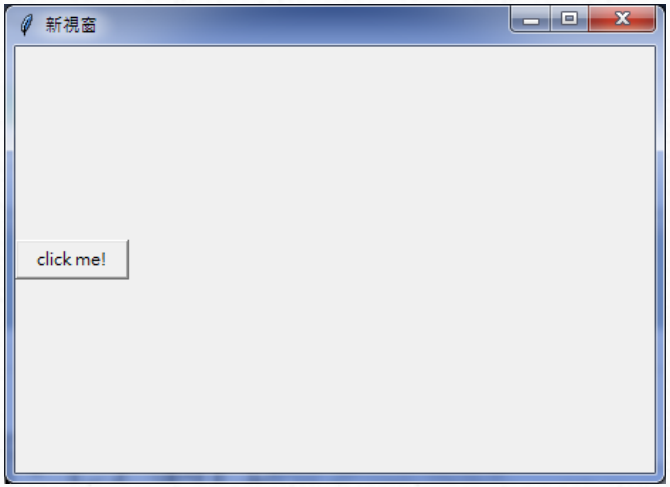
# Button元件放置

- 範例1:

```
from tkinter import *#載入函式庫
root = Tk()#建立一個視窗
root.title("新視窗")#設定視窗名稱
#設定顯示的字串、按鈕大小、靠左對齊且建立按鈕
Button(root,text="click me!",width=10).pack(side="left")
#root.geometry("長度x寬度+視窗放置的水平座標+視窗放置的垂直座標")
root.geometry("450x300+1000+0")
root.mainloop()#執行
```

# Button元件放置

- 執行結果:



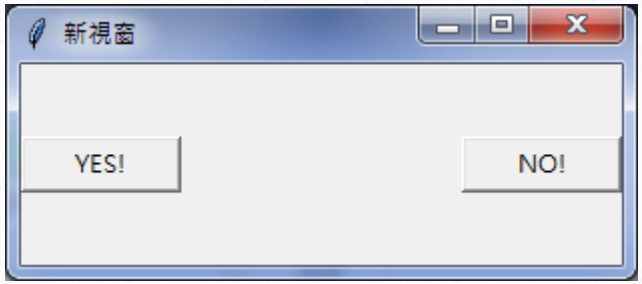
# Button元件放置

- 範例2:

```
from tkinter import *#載入函式庫
root = Tk()#建立一個視窗
root.title("新視窗")#設定視窗名稱
#設定顯示的字串、按鈕大小、靠左對齊且建立按鈕
Button(root,text="YES!",width=10).pack(side="left")
#設定顯示的字串、按鈕大小、靠右對齊且建立按鈕
Button(root,text="NO!",width=10).pack(side="right")
#root.geometry("長度x寬度+視窗放置的水平座標+視窗放置的垂直座標")
root.geometry("300x100+1000+0")
root.mainloop()#執行
```

# Button元件放置

- 執行結果:



- 以上範例，只有建立元件，沒有任何功能，以下範例，介紹如何搭配command參數，呼叫函式，達到您的使用目的。

# 透過Button呼叫函式

- 語法:

```
#將被呼叫的函式
def b1_function ():
    print("YES")
#呼叫指定函式且建立按鈕
Button(root,text="YES!",width=10,command=b1_function).pack(side="left")
```

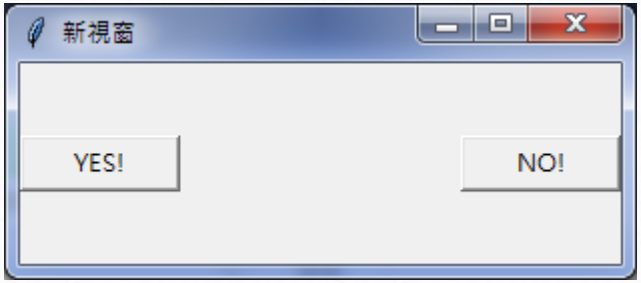
# 透過Button呼叫函式

- 範例:

```
def b1_function ():
    print("YES")
def b2_function ():
    print("NO")

from tkinter import *#載入函式庫
root = Tk()#建立一個視窗
root.title("新視窗")#設定視窗名稱
#設定顯示的字串與按鈕大小且建立按鈕
Button(root,text="YES!",width=10,command=b1_function).pack(side="left")
#設定顯示的字串與按鈕大小且建立按鈕
Button(root,text="NO!",width=10,command=b2_function).pack(side="right")
#root.geometry("長度x寬度+視窗放置的水平座標+視窗放置的垂直座標")
root.geometry("300x100+1000+0")
root.mainloop()#執行
```

# 透過Button呼叫函式

- 執行結果:




- 按下YES後:

YES

- 按下NO後:

NO

- 以上範例都是自己建立按鍵，以下介紹for迴圈搭配串列自動建立按鈕的例子。

# 設置Button字串位置

- 語法:

```
Button(root,#視窗名稱
text ="字串顯示在上方",#顯示字串
anchor = n,#字串放置的方位
width = 10,#按鍵寬度
height = 4).pack() #按鍵高度
```

# 設置Button字串位置

- anchor：指定的text或圖像(bitmap/image)在Button中的顯示位置，可用的值:e/w/n/s/ne/se/sw/nw/center 。

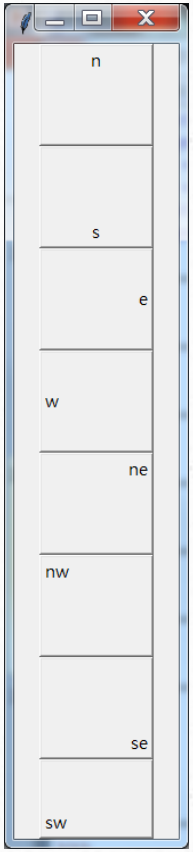
# 設置Button字串位置

- 範例:

```
from tkinter import *#載入函式庫
root = Tk()#建立一個視窗
anchor=["n", "s", "e", "w", "ne", "nw", "se", "sw"]
for a in range(0,8):
    Button(root,#視窗名稱
    text = anchor[a],#顯示字串
    anchor = anchor[a],# 字串放置的方位
    width = 10,#按鍵寬度
    height = 4).pack() #按鍵高度
#root.geometry("長度x寬度+視窗放置的水平座標+視窗放置的垂直座標")
root.geometry("100x700+0+0")
root.mainloop()#執行
```

# 設置Button字串位置

- 執行結果:



# 設置Button邊框粗細

- 語法:

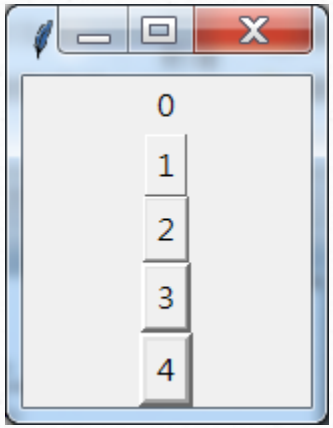
```
Button(root,
    text = "bordwidth ",#顯示數字
    bd = 2).pack()#邊框粗細設置
```

- 範例:

```
from tkinter import *#載入函式庫
root = Tk() #建立一個視窗
for bordwidth in [0,1,2,3,4]:#建立五個按鈕
    Button(root,
    text = bordwidth,#顯示數字
    bd = bordwidth).pack()#邊框粗細設置
root.mainloop() #執行
```

# 設置Button邊框粗細

- 執行結果:



# 設置Button狀態

- 語法:

```
Button(root, text = "click", state = "disabled").pack()
```

- 其他可用狀態指令:
  1. normal
  2. active
  3. disabled

# 設置Button狀態

- 範例:

```
from tkinter import *#載入函式庫
root = Tk() #建立一個視窗
def statePrint():#會被呼叫的函式
    print ("state")
for r in ["normal", "active", "disabled"]:# normal為預設且設定為active，disabled按鈕設定為不可按
    Button(root, text = r, state = r, width = 30, command = statePrint).pack(side="left")
root.mainloop()
```

- 執行結果:

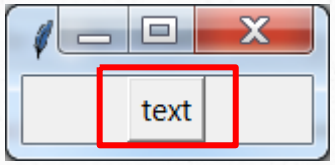
# Button動態顯示

- 範例:

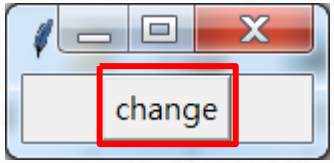
```
from tkinter import *#載入函式庫
root = Tk() #建立一個視窗
def changeText():#會被呼叫的函式
    if b["text"] == "text":
        v.set("change") #變動字串變數設為change
        print ("change")
    else:
        v.set("text") #變動字串變數設為text
        print ("text")
v = StringVar()#變動字串變數
#按鈕顯示字串為變動字串變數v(text or change )
b = Button(root ,textvariable = v,command = changeText)
v.set("text")
b.pack()
root.mainloop()
```

# Button動態顯示

- 執行結果:
  - 顯示的視窗




  - 按下text後:




  - 按下text後顯示的結果:

```
>>>
change
```

  - 按下change後顯示的結果:

```
>>>
change
text
```

# Button動態顯示

- Button使用範圍非常廣泛，在填寫表單時，經常看到當表格還未全部完成，不讓使用者按下Button鈕，以下範例，介紹如何動態改變Button 狀態。

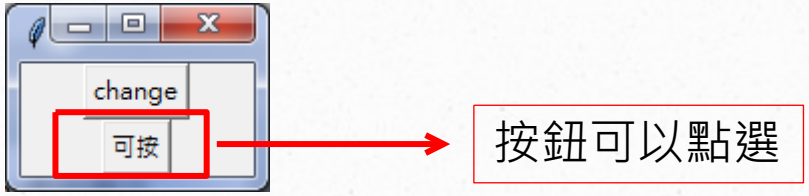
# Button動態顯示

- 範例:

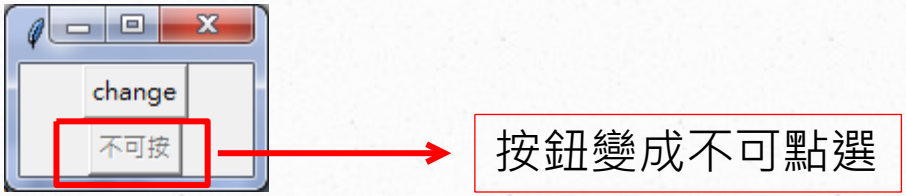
```
from tkinter import *#載入函式庫
root= Tk()
e = StringVar()#變動文字變數
e.set("可按")#預設為可按
count=0#整數變數當計數器
#自行定義的函式
def click():
  #使用全域的count整數變數
  global count
  #不是0就是1
  if(count==0):
    #如果是0將Button2設為不可以按
    Button2.config(state="disable")
    e.set("不可按")
    count=count+1
  else:
    #如果是1將Button2設為不可以按
    Button2.config(state="active")
    e.set("可按")
    count=0
#顯示字串與設定呼叫指定函式
Button1 = Button(root, text="change",command= click)
Button1.pack()
#顯示字串
Button2 = Button(root, textvariable = e)
Button2.pack()
root.mainloop()
```

# Button動態顯示

- 執行結果:
  - 預設狀態:




  - 按下change鈕:

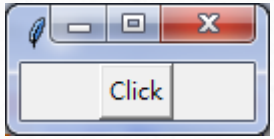


# 使Button消失

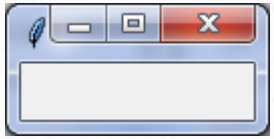
```
from tkinter import *#載入函式庫
#函式，使Button消失
def hide_me(event):
    event.widget.pack_forget()
root = Tk()#建立視窗
b1=Button(root, text="Click")#顯示字串
b1.bind("<Button-1>", hide_me)#事件，當按下滑鼠
#左鍵一下，呼叫函式
b1.pack()#建立Button
root.mainloop()
```

# 使Button消失

- 執行結果:
  - 預設狀態:




  - 按下Click後結果:



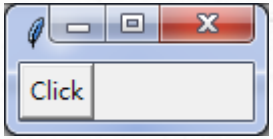
# 使Button消失

- 範例:

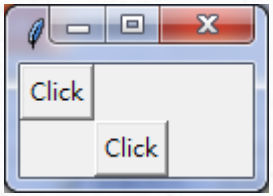
```
from tkinter import *#載入視窗程式函式庫
from random import randint #載入亂數函式庫

#隱藏原本Button，在建立兩個新的Button
def hide_me(event):
    X=randint(0,4)#亂數產生0~4的數值
    Y=randint(0,4)#亂數產生0~4的數值
    X1=randint(0,4)#亂數產生0~4的數值
    Y1=randint(0,4)#亂數產生0~4的數值
    b1=Button(root, text="Click")#顯示字串
    b1.bind("<Button-1>", hide_me)#事件，當按下#滑鼠左鍵一下，呼叫函式
    b1.grid(row = X,column = Y)#建立Button且隨機擺放

    b2=Button(root, text="Click")#顯示字串
    b2.bind("<Button-1>", hide_me)#事件，當按下滑鼠左鍵一下，呼叫函式
    b2.grid(row = X1,column = Y1)#建立Button且隨機擺放
    event.widget.grid_forget()#使原本的Button消失


root = Tk()#建立視窗
b1=Button(root, text="Click")#顯示字串
b1.bind("<Button-1>", hide_me)#事件，當按下#滑鼠左鍵一下，呼叫函式
b1.grid(row = 0,column = 0)#建立Button
root.mainloop()
```

# 使Button消失

- 執行結果:
  - 預設狀態:




  - 按下Click後:



# 資料來源

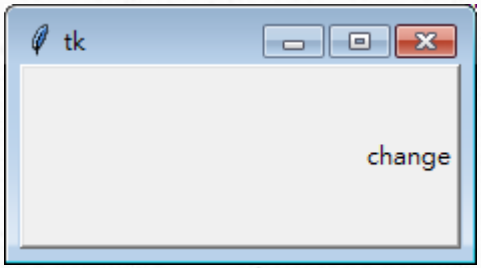
- http://pan.baidu.com/share/link?shareid=2190696726&uk=338543583&fid=714357500588689
- http://blog.csdn.net/a359680405/article/category/2799089/2

# 作業

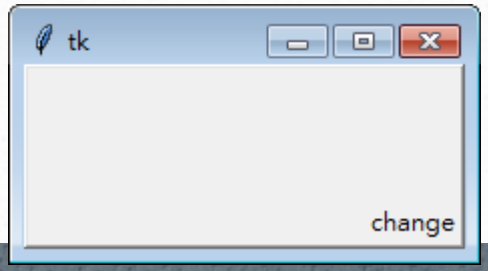
- Button大小設為 width = 30， height = 5 ，當按下Button， 字串顯示方位順序為e, se, s, sw, w, nw, n, ne, center， 最後會回到方位e。(可以透過 Button1.config(anchor="w") 改變位置)

# 作業

- 執行結果:
  - 預設狀態:




  - 按一下Button後:



```
from tkinter import *#載入函式庫
root= Tk()
count=1#整數變數當計數器
loc=["e","se","s","sw","w","nw","n","ne","center"]


#自行定義的函式
def click():
    global count
    Button1.config(anchor=loc[count])
    count+=1
    if(count>8):
        count=0

#顯示字串與設定呼叫指定函式
Button1 = Button(root,anchor="e",text="change",command= click,width = 30,height = 5)
Button1.pack()
root.mainloop()
```